Making decisions with if / else

🎯 Today's goal

By the end of this hour you will:

- ask yes/no questions with **comparisons** like `>`, `==` and `!=`
- run code only when something is true, with `if` and `else`
- choose between several options with `elif`, for example grade bands
- combine questions with `and`, `or` and `not`
- know why **indentation** (spaces at the start of a line) matters in Python

🤔 Why this matters

Until now Python ran every line, every time, from top to bottom. Real programs need to **choose**. A thermometer app shows a warning *only if* you have a fever. A shop gives a discount *only if* you are a student.

Think of a fork in a road with a sign: "if it's raining, take the left path, otherwise take the right." In data work, decisions are everywhere: labelling a temperature as cold, mild or hot, flagging a strange value as an error, or putting customers into groups. Even a machine learning model, in the end, makes a decision like "spam or not spam".

⚖️ Asking yes/no questions

On Day 2 you saw that a comparison gives a **boolean**: `True` or `False`. Here are all the comparison operators:

Operator	Question it asks	Example	Answer
`==`	equal to?	`5 == 5`	`True`
`!=`	not equal to?	`5 != 3`	`True`
`>`	greater than?	`2 > 7`	`False`
`<`	less than?	`2 < 7`	`True`
`>=`	greater than or equal to?	`7 >= 7`	`True`
`<=`	less than or equal to?	`8 <= 7`	`False`

```
temp = 38.6      # a made-up body temperature in °C

print(temp > 38)
print(temp == 37)
print(temp != 37)
print("Cairo" == "cairo")    # text must match exactly, capitals included
```

```
True
False
True
False
```

Two equals signs. One `=` means "store" (Day 2). Two `==` means "are these equal?". Mixing them up is the most common beginner mistake today.

🚦 Your first `if`

An `if` statement runs some lines **only when** a condition is `True`. The pattern is: the word `if`, a condition, a colon `:`, and then the lines to run, each pushed in by **4 spaces**:

```
temp = 38.6

if temp > 38:
    print("Fever! Rest and drink water.")
    print("Measure again in an hour.")

print("Check finished.")     # not pushed in: this always runs
```

```
Fever! Rest and drink water.
Measure again in an hour.
Check finished.
```

Change `temp` to `36.9` and run the box again. The two fever lines are skipped, but "Check finished." still prints.

The pushed-in lines are called a **block**, and the spaces in front of them are called **indentation**. In Python, indentation isn't decoration: it's how Python knows which lines belong to the `if`. Use 4 spaces (in the boxes on this page, the Tab key types 4 spaces for you). Forget them, and Python complains:

💡 This example has a mistake on purpose. Run it and read the error.

```
if temp > 38:
print("Fever!")
```

> ⚠️ **IndentationError**
>
> The spaces at the start of a line are not right.
>
> ```
>   line 2
>     print("Fever!")
>     ^^^^^
> IndentationError: expected an indented block after 'if' statement on line 1
> ```

expected an indented block means: "after the colon I expected some pushed-in lines, but found none." Add 4 spaces before `print` and it works.

🔀 Otherwise: `else`

`else` gives the other path: the lines to run when the condition is `False`. Exactly one of the two blocks runs, never both:

```
temp = 36.9

if temp > 38:
    status = "fever"
else:
    status = "normal"

print(f"Temperature {temp}: {status}")
```

```
Temperature 36.9: normal
```

Notice the idea here: the decision **stores a label in a variable** (`status`), and then we use it. Turning numbers into labels like this is something data scientists do all the time. `else` has no condition of its own, and it lines up with its `if`.

🪜 Several options: `elif`

When there are more than two paths, add `elif` (short for "else if"). Python checks the conditions **from top to bottom** and runs the **first** one that is `True`, then skips the rest. Here are grade bands for a made-up exam:

```
score = 74

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 50:
    grade = "D"
else:
    grade = "F"

print(f"A score of {score} gets grade {grade}")
```

```
A score of 74 gets grade C
```

A score of 74 is also `>= 50`, but Python never gets there: `score >= 70` was already true, so it stopped. That's why **the order matters**. Watch what happens if the checks are written the wrong way round:

```
score = 95

if score >= 50:          # wrong order: 95 is also >= 50, so this wins
    grade = "D"
elif score >= 90:
    grade = "A"
else:
    grade = "F"

print(f"A score of {score} gets grade {grade}   (oops!)")
```

```
A score of 95 gets grade D   (oops!)
```

Rule of thumb: with "greater than" bands, check the **biggest** band first. With "less than" bands, check the **smallest** first.

🔗 Combining questions: `and`, `or`, `not`

Word	True when…	Example
`and`	**both** sides are true	`age >= 18 and has_ticket`
`or`	**at least one** side is true	`is_student or age >= 65`
`not`	flips True ↔ False	`not is_member`

Here are the (made-up) discount rules of a museum:

```
age = 22
is_student = True
is_member = False

if age < 12 or age >= 65:
    discount = 50          # children and seniors: 50% off
elif is_student and age < 26:
    discount = 30          # young students: 30% off
else:
    discount = 0

if not is_member:
    print("Tip: members get free coffee!")

print(f"Your discount: {discount}%")
```

```
Tip: members get free coffee!
Your discount: 30%
```

Each side of `and` / `or` must be a complete question. Python reads `age >= 18 and <= 30` as nonsense; write `age >= 18 and age <= 30`. Python also lets you write a range the maths way: `18 <= age <= 30`.

📋 Decisions with lists and dictionaries

Yesterday's `in` gives `True` or `False`, so it fits perfectly inside an `if`. A made-up patient's allergies, and a check before giving a medicine:

```
allergies = ["penicillin", "peanuts"]
medicine = "penicillin"

if medicine in allergies:
    print(f"STOP: the patient is allergic to {medicine}.")
else:
    print(f"OK to give {medicine}.")

prices = {"apple": 2.5, "banana": 1.2}
fruit = "mango"
if fruit in prices:
    print(f"{fruit} costs {prices[fruit]}")
else:
    print(f"Sorry, we don't sell {fruit}.")   # no KeyError this time
```

```
STOP: the patient is allergic to penicillin.
Sorry, we don't sell mango.
```

You can also put an `if` **inside** another `if`. This is called **nesting**. Each level is pushed in 4 more spaces:

```
temp = 39.4
age = 70

if temp > 38:
    print("Fever.")
    if age >= 65:
        print("Over 65 with a fever: call the doctor today.")
    else:
        print("Rest and check again tomorrow.")
else:
    print("No fever.")
```

```
Fever.
Over 65 with a fever: call the doctor today.
```

Keep it flat when you can. Two levels are fine. If you find yourself four levels deep, `and` or `elif` usually makes it simpler.

🐞 The = vs == mistake

Writing one equals sign inside an `if` means "store", which makes no sense as a question. Python stops before running anything, with a `SyntaxError`, and even guesses what you meant:

💡 This example has a mistake on purpose. Run it and read the error.

```
grade = "A"
if grade = "A":
    print("Top marks!")
```

> ⚠️ **SyntaxError**
>
> Python couldn't read this code. Look for a missing quote, bracket, colon or comma near the line shown.
>
> ```
>   line 2
>     if grade = "A":
>        ^^^^^^^^^^
> SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
> ```

Read the end of the message: *Maybe you meant '==' ...*. Yes, we did. Change it to `==`.

🫙 Empty means False

A small but handy fact: in an `if`, an **empty** list `[]`, empty text `""` and the number `0` count as `False`. Anything else counts as `True`. So `if queue:` reads as "if the queue has anyone in it":

```
queue = []

if queue:
    print("Next patient:", queue[0])
else:
    print("Nobody is waiting.")

name = ""
if not name:
    print("The name is missing. Please fill it in.")
```

```
Nobody is waiting.
The name is missing. Please fill it in.
```

⚠️ Common mistakes

- Using `=` instead of `==` in a condition → `SyntaxError`.
- Forgetting the colon `:` at the end of the `if`, `elif` or `else` line.
- Missing or uneven indentation → `IndentationError`. Use 4 spaces for every level.
- Putting `elif` bands in the wrong order, so an early band catches everything.

- Writing `if x == 1 or 2:`. It's always true! Write `if x == 1 or x == 2:` (or `if x in [1, 2]:`).

✅ Recap

- Comparisons `== != < > <= >=` give `True` or `False`.
- `if condition:` runs the indented block only when the condition is true.
- `else:` is the other path; `elif condition:` adds more paths. Only the first true one runs.
- `and` needs both, `or` needs at least one, `not` flips.
- `in` works inside conditions: `if medicine in allergies:`
- Indentation (4 spaces) shows which lines belong to which block.
- Empty lists, empty text and 0 count as `False`.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

Today the checker is extra curious: it also tries your code with *other* values in the first lines. So keep those lines as they are, and let `if` make the decision instead of typing the answer.

1. Pass or fail

An exam is passed with a score of **50 or more**. Using `if` and `else`, store `"pass"` or `"fail"` in a variable called `result`.

```
score = 72

# Decide: result is "pass" or "fail"
```

Hints:

1. "50 or more" is `score >= 50`. Don't forget the colon at the end of the line.
2. The lines inside `if` and `else` need 4 spaces in front.
3. `if score >= 50:` → `result = "pass"` → `else:` → `result = "fail"`

2. Cold, mild or hot

Store a label for the air temperature `temp` in a variable called `label`, using `if`, `elif` and `else`:

- below 10 → `"cold"`
- from 10 up to (but not including) 25 → `"mild"`
- 25 or more → `"hot"`

```
temp = 16

# Decide the label
```

Hints:

1. With "less than" bands, check the smallest first: `if temp < 10:`
2. The next band only needs `elif temp < 25:`, because anything below 10 was already caught.
3. Everything left over is hot: `else:` → `label = "hot"`

3. Cinema ticket

A (made-up) cinema has these prices. Store the right one in a variable called `price`:

- children under 12 → `5`
- otherwise, seniors (65 or older) **or** students → `7`
- everyone else → `10`

```
age = 67
is_student = False

# Decide the price
```

Hints:

1. Check children first: `if age < 12:`
2. The second rule has two questions joined by `or`: `elif age >= 65 or is_student:`
3. Finish with `else:` → `price = 10`

4. BMI category

The starting code calculates a BMI. Store its category in a variable called `category`:

BMI	category
below 18.5	`"underweight"`
18.5 up to (not including) 25	`"normal"`
25 up to (not including) 30	`"overweight"`
30 or more	`"obese"`

(Made-up person. BMI is a teaching example here, not medical advice.)

```
weight = 82
height = 1.78
bmi = weight / height ** 2

# Decide the category
```

Hints:

1. Compare `bmi` (not the weight). Start with the smallest band: `if bmi < 18.5:`
2. Then `elif bmi < 25:` and `elif bmi < 30:`, and `else` for the rest.
3. Each band stores a word: `category = "normal"` and so on.

5. Challenge: leap year

A year is a **leap year** (it has a 29 February) when it can be divided by 4, **except** years that can be divided by 100, which are only leap years if they can also be divided by 400. So 2024 is a leap year, 1900 is not, and 2000 is.

Store `True` or `False` in a variable called `is_leap`.

Reminder from Day 1: `%` gives the remainder of a division. "`year` can be divided by 4" is written `year % 4 == 0`.

```
year = 2024

# Decide: is_leap is True or False
```

Hints:

1. Check the rarest rule first: `if year % 400 == 0:` → leap year.
2. Then `elif year % 100 == 0:` → not a leap year, then `elif year % 4 == 0:` → leap year, `else` → not.
3. Or all in one line: `is_leap = year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)`

➡️ Tomorrow

Repeating work with loops: today you labelled *one* temperature. But what about a whole year of them? Tomorrow you'll learn `for` and `while` loops, which repeat code for every item in a list, so you can add up a week of sales or count the hot days, however long the list is.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Pass or fail

```
score = 72

if score >= 50:
    result = "pass"
else:
    result = "fail"
```

2. Cold, mild or hot

```
temp = 16

if temp < 10:
    label = "cold"
elif temp < 25:
    label = "mild"
else:
    label = "hot"
```

3. Cinema ticket

```
age = 67
is_student = False

if age < 12:
    price = 5
elif age >= 65 or is_student:
    price = 7
else:
    price = 10
```

4. BMI category

```
weight = 82
height = 1.78
bmi = weight / height ** 2

if bmi < 18.5:
    category = "underweight"
elif bmi < 25:
    category = "normal"
elif bmi < 30:
    category = "overweight"
else:
    category = "obese"
```

5. Challenge: leap year

```
year = 2024

if year % 400 == 0:
    is_leap = True
elif year % 100 == 0:
    is_leap = False
elif year % 4 == 0:
    is_leap = True
else:
    is_leap = False
```